

Міністерство Освіти і Науки України
Національний Університет “Львівська Політехніка”



Кафедра ЕОМ

**ВИКОРИСТАННЯ ФУНКЦІОНАЛЬНОЇ ДЕКОМПОЗИЦІЇ ДЛЯ
РОЗВ’ЯЗКУ ОБЧИСЛЮВАЛЬНИХ ЗАДАЧ**

Методичні вказівки

до лабораторної роботи з курсу “Паралельні та розподілені обчислення”
для студентів спеціальності F7 “Комп’ютерна інженерія”

Затверджено
на засіданні кафедри
”Електронні обчислювальні машини”
Протокол № _ від 2026 року

Львів – 2026

МЕТА РОБОТИ. Вивчити методи декомпозицій задач. Набути навиків розв'язування задач з використанням функціональної декомпозиції.

ТЕОРЕТИЧНІ ВІДОМОСТІ

Найважливішим та найважчим етапом при створенні програми є розробка алгоритму, особливо, якщо мова йде про паралельний алгоритм. Процес створення паралельного алгоритму можна розбити на чотири кроки.

1. Декомпозиція. На цьому етапі вихідна задача аналізується, оцінюється можливість її розпаралелювання. Іноді вигаши від розпаралелення може бути незначним, а трудоемкість розробки паралельної програми велика. В цьому випадку перший крок розробки алгоритму виявляється і останнім. Якщо ж ситуація відмінна від описаної, то задача та пов'язані з нею дані розділяються на дрібніші частини – підзадачі і фрагменти структур даних. Особливості архітектури конкретної обчислювальної системи на цьому етапі можуть не враховуватися.

2. Проектування комунікацій(обміну даними) між задачами. На цьому етапі визначаються зв'язки, необхідні для пересилання вхідних даних, проміжних результатів виконання підзадач, а також комунікації, що необхідні для керування роботою під задач. Обираються методи та алгоритми комунікацій.

3. Укрупнення. Підзадачі можуть об'єднуватися у більші блоки, якщо це дозволяє підвищити ефективність алгоритму і знизити трудоемкість розробки. Основними критеріями на даному кроці є ефективність алгоритму (в першу чергу-продуктивність) та трудоемкість його реалізації.

4. Планування обчислень. На цьому кроці виконується розподіл під задач між процесорами. Основний критерій вибору способу розміщення під задач – ефективне використання процесорів з мінімальними затратами часу на обмін даними.

Декомпозиція (сегментування).

На цьому етапі визначається степінь можливого розпаралелення задачі. Іноді декомпозиція поставленої задачі природним чином впливає з самої задачі тому є очевидною, іноді – ні. Чим менший розмір підзадач, отриманих в результаті декомпозиції, чим більша їх кількість, тим гнучкішим виявляється паралельний алгоритм і тим легше забезпечити рівномірне завантаження процесорів обчислювальної системи. Надалі, можливо доведеться укрупнити алгоритм, оскільки слід прийняти до уваги інтенсивність обміну даними та інші фактори. Сегментувати можна як обчислювальний алгоритм, так і дані. Застосовуються різні варіанти декомпозиції.

В методі декомпозиції даних спочатку сегментуються дані, а потім алгоритм їх обробки. Дані розбиваються на сегменти приблизно однакового розміру. З

фрагментами даних пов'язуються операції їх обробки, з яких і формуються підзадачі. Визначаються необхідні правила обміну даними. Перекриття частин, на які розбивається задача, повинне бути мінімальним. Це дозволяє уникнути дублювання обчислень. Схема розбиття може в подальшому уточнюватися. Іноді, якщо це необхідно для зменшення кількості обмінів, допускається збільшення степені перекриття фрагментів задачі.

При декомпозиції даних спочатку аналізуються структури даних найбільшого розміру, або ті, до яких найчастіше відбувається звертання. На різних стадіях розрахунків можуть використовуватися різні структури даних, тому можуть бути необхідними динамічні, тобто такі, що міняються в часі схеми декомпозиції цих структур.

В методі функціональної декомпозиції спочатку сегментується обчислювальний алгоритм, а потім під цю схему підбирається схема декомпозиції даних. Успіх у цьому випадку не завжди гарантовано, оскільки схема може вимагати багатьох додаткових пересилань даних. Цей метод декомпозиції може виявитися корисним у випадку, якщо немає структур даних, які б могли бути розпаралелені очевидним чином. Функціональна декомпозиція відіграє важливу роль і як метод структурування програм. Вона може виявитися корисною при моделюванні складних систем, що складаються з множини простих підсистем, зв'язаних між собою набором інтерфейсів.

Розмір блоків, з яких складається паралельна програма, може бути різним. В залежності від розміру блоків, алгоритм може мати різну “зернистість”. Її виміром в найпростішому випадку є кількість операцій у блоці. Виділяють три степені зернистості: дрібнозернистий, середньоблоковий та великоблоковий. Зернистість пов'язана з рівнем паралелізму.

Паралелізм на рівні команд найбільш дрібнозернистий. Його масштаб менше ніж 20 команд на блок. Кількість підзадач, що паралельно виконуються – від однієї до кількох тисяч, при чому середній масштаб паралелізму складає біля п'яти команд на блок.

Наступний рівень - паралелізм на рівні циклів. Переважно, цикл містить не більше 500 команд. Якщо ітерації циклу незалежні, вони можуть виконуватися, наприклад за допомогою конвеєра або на векторному процесорі. Це також дрібнозернистий паралелізм.

Паралелізм на рівні підзадач – середньоблоковий. Розмір блоку – до 2000 команд. Виконання такого паралелізму реалізувати важче, оскільки слід враховувати можливі міжпроцедурні залежності. Вимоги до комунікацій менші, ніж у випадку паралелізму на рівні команд.

Паралелізм на рівні програм (задач) – великоблоковий. Він означає виконання незалежних програм на паралельному комп'ютері. Великоблоковий паралелізм повинен підтримуватися операційною системою.

Обмін даними через спільні/розподілені змінні використовується на рівнях дрібнозернистого і середньоблокового паралелізму, а на великоблоковому – засобами передачі повідомлень.

Досягнути ефективності роботи паралельної програми можна, якщо збалансувати зернистість алгоритму і затрати часу на обмін даними.

Частини програми можуть виконуватися паралельно, лише якщо вони незалежні.

Незалежність за даними полягає в тому, що дані, які обробляються однією частиною програми не модифікуються іншою її частиною.

Незалежність за керуванням полягає в тому, що послідовність виконання частин програми може бути визначена лише під час виконання програми(наявність залежності по виконанню наперед визначає порядок виконання).

Незалежність за ресурсами забезпечується достатньою кількістю комп'ютерних ресурсів(об'єм пам'яті, кількість функціональних вузлів та ін.).

Залежність за виводом виникає, якщо дві підзадачі виконують запис в одну і ту ж змінну. А *залежність за вводом/виводом*, якщо оператори вводу/виводу двох чи декількох під задач звертаються до одного файлу(чи змінної).

Дві підзадачі можуть виконуватися паралельно, якщо вони незалежні за даними, за керуванням і за операціями вводу виводу.

ЗАВДАННЯ

1. Використовуючи метод функціональної декомпозиції, розробити алгоритм обчислення запропонованого матрично-векторного виразу, який би враховував можливість паралельного виконання і був оптимальним з точки зору часових затрат.
2. На основі створеного алгоритму, використовуючи засоби MPI, написати програму, яка дозволяє обчислити вираз та ілюструє проведену декомпозицію для оптимальної кількості процесорів.
3. Дослідити ефективність паралельного обчислення за критерієм часу.

ПОРЯДОК ВИКОНАННЯ РОБОТИ

1. Проаналізувати, наведені нижче, правила обрахунку елементів виразу.
2. Провести декомпозицію задачі, виходячи з можливості паралельного виконання окремих підзадач для заданої кількості процесорів.
3. Об'єднати отримані проміжні результати і обрахувати кінцевий вираз.
4. Написати послідовну програму обчислення виразу.
5. Написати і продемонструвати програму обчислення виразу засобами MPI.
6. Визначити паралелізм якого рівня присутній в алгоритмі та зробити висновки щодо залежностей даних, керування, ресурсів, вводу/виводу.
7. Дослідити час обчислення виразу в залежності від розмірності вхідних даних.
8. Скласти звіт про виконану роботу.

ЗМІСТ ЗВІТУ

1. Тема, мета, аналіз завдання (згідно варіанту).
2. Схема декомпозиції задачі та коментарі до неї.
3. Тексти програм та результат їх роботи на довільному наборі вхідних даних, для розмірності $n > 3$.
4. Дослідження ефективності паралельної програми у порівнянні з послідовною за часом виконання.
5. Висновки.

Вагомі зауваження

1. Окрім безпосередніх обчислень, програма повинна мати інтерфейс користувача, який забезпечує:
 - 1) ввід(з клавіатури) розмірності даних (n);
 - 2) можливість вибору – ввід даних(тобто елементів матриці та векторів) з клавіатури чи генерування їх випадковим чином;
 - 3) вивід на екран (або у файл) проміжних результатів за потребою користувача;
 - 4) обов'язковий вивід остаточних результатів на екран і у файл у зрозумілому вигляді.
2. Всі вхідні дані є цілими числами, більшими за нуль.

ВАРІАНТИ ЗАВДАНЬ

Задано (вхідні дані):

y_1 і y_2 – вектори-стовпці з n елементів,

Y_3 – квадратна матриця порядку n .

y^T позначає операцію транспонування.

1	$x = Y_3^3 * y_2 + (y_1 * y_2^T)^2 * Y_3 * (y_1 + y_2)$ стовпець
2	$x = Y_3^2 * y_2 + (y_1 * y_2^T)^3 * y_2 + Y_3^3 * y_1$ стовпець
3	$x = Y_3 * y_2 * y_2^T + (y_2 * y_1^T)^2 + Y_3^2 * y_1 * y_1^T$ матриця
4	$x = Y_3 * y_1 + Y_3^3 * y_2 + (y_2 * y_1^T * Y_3)^2 * y_1$ стовпець
5	$x = (y_1^T * Y_3^3 + y_2^T) * (Y_3 * y_2 + y_1 * y_2^T * Y_3^2 * y_2)$ число
6	$x = y_2^T * y_1 * Y_3^2 + (y_1 * y_2^T)^2 * Y_3 + y_2^T * Y_3 * y_1 * Y_3$ матриця
7	$x = y_2^T * ((y_1 * y_2^T)^2 + Y_3^3 + y_1^T * y_2 * Y_3)$ рядок
8	$x = ((y_1 * y_2^T)^2 * Y_3 + Y_3^2 + y_1 * y_2^T) * (Y_3 * y_2 + y_1)$ стовпець
9	$x = y_2^T * (Y_3 + (y_1 * y_2^T)^2) + y_1^T * (Y_3 + Y_3^3)$ рядок
10	$x = (y_1^T * (y_2 * y_2^T)^2 + y_1^T * Y_3^3) * (Y_3 * y_1 + y_2 * y_2^T * y_1)$ число
11	$x = y_2 * y_1^T + y_2^T * y_2 * Y_3 + Y_3^3 + Y_3 * y_1 * y_1^T * Y_3$ матриця
12	$x = Y_3^2 + y_1^T * (Y_3^2 * y_1 + y_2) * Y_3 + (y_1 * y_2^T)^3$ матриця
13	$x = Y_3^2 * y_2 * y_2^T * y_1 + Y_3^2 * y_2 + (y_1 * y_2^T)^2 * y_1$ стовпець
14	$x = Y_3^3 * y_1 * y_1^T + (y_2 * y_1^T)^3 + y_2^T * Y_3 * y_1 * Y_3$ матриця
15	$x = y_2^T + y_1^T + (Y_3^2 * y_1)^T + y_2^T * (y_2 * y_1^T)^2$ рядок
16	$x = (Y_3 * y_2 + y_2)^T * (Y_3^2 y_2 + (y_1 * y_2^T)^2 * y_1)$ число
17	$x = Y_3^3 + y_2^T * y_1 * Y_3 + (y_1 * y_2^T)^2 + Y_3 * y_2 * y_1^T$ матриця
18	$x = (y_2 * y_1^T)^2 + Y_3^3 + Y_3^2 + Y_3 * y_2 * y_1^T$ матриця
19	$x = y_1^T * ((y_2 * y_1^T)^2 + Y_3^2 + y_2^T * y_1 * Y_3) * y_2$ число

20	$x = (y_1^T * (y_2 * y_2^T)^3 + y_2 * Y_3^3 + y_2^T * Y_3) * y_1$ число
21	$x = (Y_3^2 * y_2 + Y_3 * y_1 + (y_2 * y_2^T)^2 * y_1)^T * Y_3^2$ рядок
22	$x = (y_1^T * (Y_3 * y_1 + y_2) * Y_3^2 + (y_2 * y_2^T)^3) * y_2$ стовпець
23	$x = Y_3 * y_1 * y_2^T + (y_1 * y_2^T)^3 + Y_3^2 + y_2 * y_2^T * Y_3$ матриця
24	$x = (y_1^T * Y_3^2 + y_2^T) * ((y_1 * y_1^T)^2 * y_2 + y_2)$ число
25	$x = (y_2^T * Y_3^3 + y_1^T)(Y_3^2 y_1 + (y_2 * y_1^T)^3 * y_2)$ число
26	$x = (y_1^T * Y_3^3 + y_2^T * Y_3) * (y_1 + y_1 * y_2^T * Y_3^2 * y_2)$ число
27	$x = y_2^T * y_1 * Y_3^2 + Y_3 * (y_1 * y_2^T) * Y_3 + Y_3^3$ матриця
28	$x = y_2^T * ((y_1 * y_2^T)^3 + Y_3^3 + (y_1 * y_2^T) * Y_3) * y_1$ число
29	$x = (Y_3^2 + (y_1 * y_2^T)^3) * (Y_3 * y_2 + Y_3^2 * y_1)$ стовпець
30	$x = y_2^T * (Y_3 + Y_3^2 + (y_2 * y_2^T)^3) + y_1^T * Y_3^3$ рядок

Приклад виконання лабораторної роботи №2

Варіант завдання:

$$x = y_2^T * ((y_1 * y_2^T)^3 + Y_3^2 + y_1^T * y_2 * Y_3) * y_1$$

Використовуючи метод функціональної декомпозиції, розробимо алгоритм обчислення запропонованого матрично-векторного виразу, який враховує можливість паралельного виконання і є оптимальним з точки зору часових затрат.

Перед тим, як розбивати задачу на паралельні частини, необхідно визначити приблизну часову складність окремих дій, щоб паралельне виконання було оптимальним. Розглянемо операції, які необхідно виконати для отримання результату:

$$T_1 = (y_1 * y_2^T) - \text{множення стовпця на рядок, часова складність } O(n^2);$$

$$T_2 = T_1 * T_1 = (y_1 * y_2^T) * (y_1 * y_2^T) = (y_1 * y_2^T)^2 - \text{множення матриці на матрицю, часова складність } O(n^3);$$

$$T_3 = T_2 * T_1 = (y_1 * y_2^T)^3 - \text{множення матриці на матрицю, часова складність } O(n^3);$$

$$T_4 = Y_3 * Y_3 = Y_3^2 - \text{множення матриці на матрицю, часова складність } O(n^3);$$

$$x_1 = y_1^T * y_2 - \text{множення рядка на стовпець, часова складність } O(n);$$

$$T_5 = x_1 * Y_3 = y_1^T * y_2 * Y_3 - \text{множення числа на матрицю, часова складність } O(n^2);$$

$$T_6 = T_4 + T_5 = Y_3^2 + y_1^T * y_2 * Y_3 - \text{додавання двох матриць, часова складність } O(n^2);$$

$$T_7 = T_3 + T_6 = (y_1 * y_2^T)^3 + Y_3^2 + y_1^T * y_2 * Y_3 - \text{додавання двох матриць, часова складність } O(n^2);$$

$$t_1 = y_1^T * T_7 = y_1^T * ((y_1 * y_2^T)^3 + Y_3^2 + y_1^T * y_2 * Y_3) - \text{множення рядка на матрицю, часова складність } O(n^2);$$

$$x = t_1 * y_1 = y_1^T * ((y_1 * y_2^T)^3 + Y_3^2 + y_1^T * y_2 * Y_3) * y_1 - \text{множення рядка на стовпець, часова складність } O(n).$$

У даному виразі є такі операції: множення стовпця на рядок, множення матриці на матрицю, множення рядка на стовпець, додавання матриць, множення числа на матрицю, множення рядка на матрицю. Найзатратніша операція – це множення матриці на матрицю, часова складність якої становить $O(n^3)$, для решти операцій часова складність становить $O(n^2)$ або $O(n)$. Різниця між множенням

матриці на матрицю і іншими операціями у часовій складності складає не менше ніж n раз.

У заданому виразі є три операції множення матриці на матрицю – отримання матриці Y_3^2 і обчислення матриці $(y_1 * y_2^T)^3$, але отримання значення $(y_1 * y_2^T)^3$ розпаралелити не вийде, тому доцільно розбити MPI програму на два процеси. Процес з рангом 0 (на схемі зображений синім кольором) вводить вхідні дані і обчислює значення $(y_1 * y_2^T)^3$, отримує від процесу з рангом 1 значення матриці $Y_3^2 + y_1^T * y_2 * Y_3$ і виконує усі подальші обчислення виразу. Процес з рангом 1 (на схемі зображений зеленим кольором) обчислює значення матриці $Y_3^2 + y_1^T * y_2 * Y_3$ і пересилає отриману матрицю процесу з рангом 0.

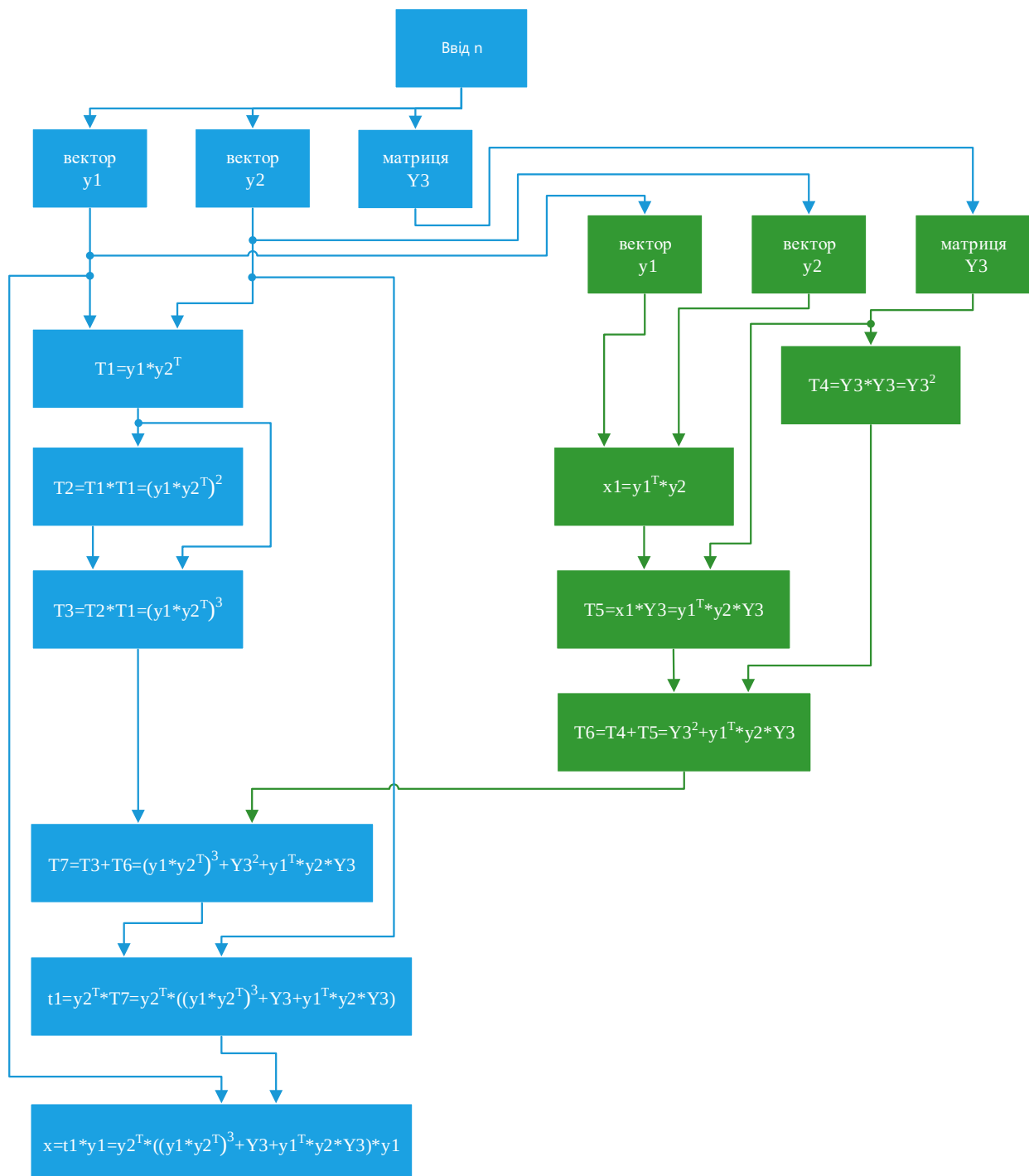


Рис. 1. Схема функціональної декомпозиції задачі

Перевіримо роботу розробленої однопоточної програми (файл Results.txt):

Vector y1 =

1.00
2.00
3.00
4.00

Vector y2 =

5.00
6.00
7.00
8.00

Matrix Y3 =

1.00	2.00	3.00	4.00
5.00	6.00	7.00	8.00
8.00	7.00	6.00	5.00
4.00	3.00	2.00	1.00

Matrix y1*t_y2 =

5.00	6.00	7.00	8.00
10.00	12.00	14.00	16.00
15.00	18.00	21.00	24.00
20.00	24.00	28.00	32.00

Matrix (y1*t_y2)*(y1*t_y2) =

350.00	420.00	490.00	560.00
700.00	840.00	980.00	1120.00
1050.00	1260.00	1470.00	1680.00
1400.00	1680.00	1960.00	2240.00

Matrix (y1*t_y2)*(y1*t_y2)*(y1*t_y2) =

24500.00	29400.00	34300.00	39200.00
49000.00	58800.00	68600.00	78400.00
73500.00	88200.00	102900.00	117600.00
98000.00	117600.00	137200.00	156800.00

Matrix Y3*Y3 =

51.00	47.00	43.00	39.00
123.00	119.00	115.00	111.00
111.00	115.00	119.00	123.00
39.00	43.00	47.00	51.00

t_y1*y2 =

70.00

Matrix t_y1*y2*Y3 =

70.00	140.00	210.00	280.00
350.00	420.00	490.00	560.00
560.00	490.00	420.00	350.00
280.00	210.00	140.00	70.00

Matrix $Y3 \cdot Y3 + t_y1 \cdot y2 \cdot Y3 =$
 121.00 187.00 253.00 319.00
 473.00 539.00 605.00 671.00
 671.00 605.00 539.00 473.00
 319.00 253.00 187.00 121.00

Matrix $(y1 \cdot t_y2) \cdot (y1 \cdot t_y2) \cdot (y1 \cdot t_y2) + Y3 \cdot Y3 + t_y1 \cdot y2 \cdot Y3 =$
 24621.00 29587.00 34553.00 39519.00
 49473.00 59339.00 69205.00 79071.00
 74171.00 88805.00 103439.00 118073.00
 98319.00 117853.00 137387.00 156921.00

Vector $t_y2 \cdot ((y1 \cdot t_y2) \cdot (y1 \cdot t_y2) \cdot (y1 \cdot t_y2) + Y3 \cdot Y3 + t_y1 \cdot y2 \cdot Y3) =$
 1725692.00 2068428.00 2411164.00 2753900.00

$x = t_y2 \cdot ((y1 \cdot t_y2) \cdot (y1 \cdot t_y2) \cdot (y1 \cdot t_y2) + Y3 \cdot Y3 + t_y1 \cdot y2 \cdot Y3) \cdot y1 =$
 24111640.00

А також роботу розробленої MPI програми (файл Results_0.txt):

Vector $y1 =$
 1.00
 2.00
 3.00
 4.00

Vector $y2 =$
 5.00
 6.00
 7.00
 8.00

Matrix $Y3 =$
 1.00 2.00 3.00 4.00
 5.00 6.00 7.00 8.00
 8.00 7.00 6.00 5.00
 4.00 3.00 2.00 1.00

Matrix $y1 \cdot t_y2 =$
 5.00 6.00 7.00 8.00
 10.00 12.00 14.00 16.00
 15.00 18.00 21.00 24.00
 20.00 24.00 28.00 32.00

Matrix $(y1 \cdot t_y2) \cdot (y1 \cdot t_y2) =$
 350.00 420.00 490.00 560.00
 700.00 840.00 980.00 1120.00
 1050.00 1260.00 1470.00 1680.00
 1400.00 1680.00 1960.00 2240.00

Matrix $(y1 \cdot t_y2) \cdot (y1 \cdot t_y2) \cdot (y1 \cdot t_y2) =$
 24500.00 29400.00 34300.00 39200.00
 49000.00 58800.00 68600.00 78400.00
 73500.00 88200.00 102900.00 117600.00
 98000.00 117600.00 137200.00 156800.00

Matrix $(y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 + t_y1*y2*Y3 =$

24621.00	29587.00	34553.00	39519.00
49473.00	59339.00	69205.00	79071.00
74171.00	88805.00	103439.00	118073.00
98319.00	117853.00	137387.00	156921.00

Vector $t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 + t_y1*y2*Y3) =$

1725692.00	2068428.00	2411164.00	2753900.00
------------	------------	------------	------------

$x = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 + t_y1*y2*Y3)*y1 =$

24111640.00

І файл Results_1.txt:

Matrix $Y3*Y3 =$

51.00	47.00	43.00	39.00
123.00	119.00	115.00	111.00
111.00	115.00	119.00	123.00
39.00	43.00	47.00	51.00

$t_y1*y2 =$

70.00

Matrix $t_y1*y2*Y3 =$

70.00	140.00	210.00	280.00
350.00	420.00	490.00	560.00
560.00	490.00	420.00	350.00
280.00	210.00	140.00	70.00

Matrix $Y3*Y3 + t_y1*y2*Y3 =$

121.00	187.00	253.00	319.00
473.00	539.00	605.00	671.00
671.00	605.00	539.00	473.00
319.00	253.00	187.00	121.00

Проведемо дослідження ефективності розпаралелювання, для цього запусимо розроблені програми декілька раз з різними значеннями n (рекомендовано запускати програму у конфігурації Release). Отримані значення збережемо у таблицю і побудуємо графіки на основі цих даних.

Параметри пристрою, на якому виконувались дослідження:

Процесор: Intel(R) Core(TM) i5-4460 CPU @3.20GHz, 4 Cores

ОЗП: 8,00 ГБ (доступно для використання: 7,88 ГБ)

Тип ОС: 64-розрядна операційна система Windows 10 22H2

Таблиця 1.

Дослідження розпаралелювання

Розмірність, n	Час виконання програми, секунди	
	Однопотокова програма	MPI програма
10	0,000018300	0,000537800
100	0,012864500	0,008819400
500	2,057802900	1,475005500
1000	39,70101640	27,43542350
1500	145,8432048	98,84066430
2000	389,6322311	253,6853689
2500	743,4884968	507,7115773

На основі отриманих даних побудуємо графіки:

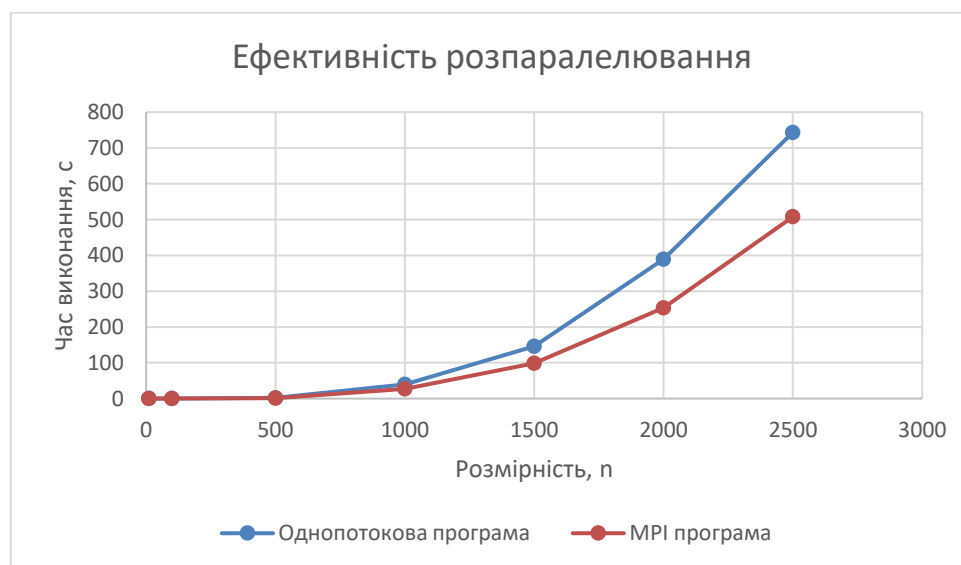
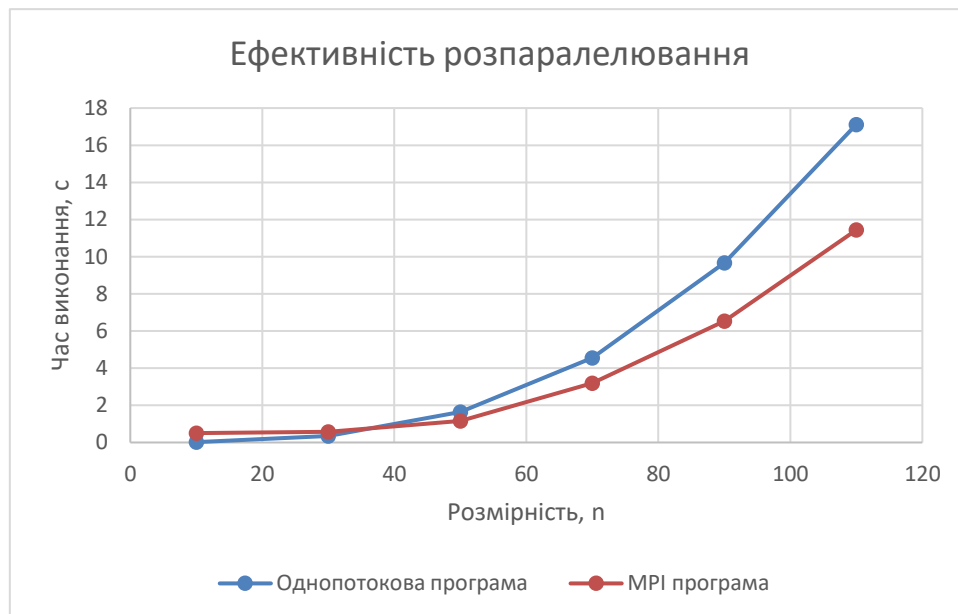


Рис. 2. Графік ефективності розпаралелювання.

Дослідження розпаралелювання при малих значеннях n

Розмірність n	Час виконання програми, мілісекунди	
	Однопотокова програма	MPI програма
10	0,018400	0,504400
30	0,354800	0,582100
50	1,644200	1,164900
70	4,552600	3,184700
90	9,666100	6,534600
110	17,11610	11,43840

Рис. 3. Графік ефективності розпаралелювання при малих значеннях n .

Висновки. Під час виконання лабораторної роботи було створено послідовну і MPI програму, які обчислюють результат заданого матричного виразу. Перед написанням MPI програми спочатку було розроблено схему декомпозиції. Аналіз схеми показав, що доцільно розбивати обчислення на два процеси, що і було зроблено.

Як видно з графіку, програма, яка виконується послідовно, є повільнішою за MPI, за винятком дуже малих значень n . MPI програма є ефективнішою за послідовну програму. Обмін даними відбувається через пересилання даних між процесами.

```

// Додаток А – однопотокова програма
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <time.h>
#include <windows.h>

double** CreateMatrix(int m, int n)
{
    double** a = (double**)malloc(m * sizeof(double*));
    for (int i = 0; i < m; i++)
        a[i] = (double*)malloc(n * sizeof(double));
    return a;
}

void DeleteMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        free(a[i]);
    free(a);
}

void FillMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            a[i][j] = rand() % 10 + 1;
            //a[i][j] = 1;
        }
}

void InputMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("Enter [%d][%d] element = ", i, j);
            scanf_s("%lf", &a[i][j]));
        }
        printf("\n");
    }
}

void PrintMatrix(double** a, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
            printf("%.2f\t", a[i][j]);
        printf("\n");
    }
    printf("\n");
}

void PrintMatrixToFile(FILE* f, char* title, double** a, int m, int n)
{
    fprintf(f, title);
    fprintf(f, "\n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
            fprintf(f, "%.2f\t", a[i][j]);
        fprintf(f, "\n");
    }
}

```

```

    }
    fprintf(f, "\n");
}

// c = a * b, a - mxn, b - nxk, c - mxk
double** MultMatrix(double** c, double** a, double** b, int m, int n, int k)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < k; j++)
        {
            c[i][j] = 0;
            for (int l = 0; l < n; l++)
                c[i][j] = c[i][j] + a[i][l] * b[l][j];
        }
    return c;
}

// b = k * a
double** MultMatrixNum(double** b, double k, double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            b[i][j] = a[i][j] * k;
    return b;
}

// c = a + b
double** SumMatrix(double** c, double** a, double** b, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    return c;
}

// vector transposition (column-to-row conversion)
void TVector(double** b, double** a, int n)
{
    for (int i = 0; i < n; i++)
        b[0][i] = a[i][0];
}

int main(void)
{
    srand((unsigned int)time(NULL));

    int n;
    printf("Enter the matrix size n : ");
    scanf_s("%d", &n);

    double** y1 = CreateMatrix(n, 1);
    double** y2 = CreateMatrix(n, 1);
    double** Y3 = CreateMatrix(n, n);

    int in;
    printf("\nSelect an input option : 1 - manual input, else - random input : ");
    scanf_s("%d", &in);
    printf("\n");

    const char* fileName = "Results.txt";
    FILE* file = fopen(fileName, "w");
    if (file == NULL)
    {
        printf("\nError opening file. Check the path and permissions !\n");
        return 1;
    }
}

```



```

if (in == 1)
{
    // manual input vector y1
    printf("Enter the elements of vector y1 :\n");
    InputMatrix(y1, n, 1);
    PrintMatrixToFile(file, (char*)"Vector y1 = ", y1, n, 1);

    // manual input vector y2
    printf("Enter the elements of vector y2 :\n");
    InputMatrix(y2, n, 1);
    PrintMatrixToFile(file, (char*)"Vector y2 = ", y2, n, 1);

    // manual input matrix Y3
    printf("Enter the elements of matrix Y3 :\n");
    InputMatrix(Y3, n, n);
    PrintMatrixToFile(file, (char*)"Matrix Y3 = ", Y3, n, n);
}
else
{
    printf("Random input... \n"); \

    // random input vector y1
    FillMatrix(y1, n, 1);
    PrintMatrixToFile(file, (char*)"Vector y1 = ", y1, n, 1);

    // random input vector y2
    FillMatrix(y2, n, 1);
    PrintMatrixToFile(file, (char*)"Vector y2 = ", y2, n, 1);

    // random input matrix Y3
    FillMatrix(Y3, n, n);
    PrintMatrixToFile(file, (char*)"Matrix Y3 = ", Y3, n, n);
}

double** t_y1 = CreateMatrix(1, n);
double** t_y2 = CreateMatrix(1, n);

double** T1 = CreateMatrix(n, n);
double** T2 = CreateMatrix(n, n);
double** T3 = CreateMatrix(n, n);
double** T4 = CreateMatrix(n, n);
double** T5 = CreateMatrix(n, n);
double** T6 = CreateMatrix(n, n);
double** T7 = CreateMatrix(n, n);

double** x1 = CreateMatrix(1, 1);

double** t1 = CreateMatrix(1, n);

double** x = CreateMatrix(1, 1);

int out;
printf("\nWrite intermediate results to a file? : 1 - yes, else - no : ");
scanf_s("%d", &out);

LARGE_INTEGER frequency, start, end;
QueryPerformanceFrequency(&frequency);
QueryPerformanceCounter(&start);

TVector(t_y1, y1, n);
TVector(t_y2, y2, n);

// calculation matrix T1 = y1*t_y2
MultMatrix(T1, y1, t_y2, n, 1, n);

// calculation matrix T2 = T1*T1 = (y1*t_y2)*(y1*t_y2)

```

```

MultMatrix(T2, T1, T1, n, n, n);

// calculation matrix T3 = T2*T1 = T1*T1*T1 = (y1*t_y2)*(y1*t_y2)*(y1*t_y2)
MultMatrix(T3, T2, T1, n, n, n);

// calculation matrix T4 = Y3*Y3
MultMatrix(T4, Y3, Y3, n, n, n);

// calculation x1 = t_y1*y2
MultMatrix(x1, t_y1, y2, 1, n, 1);

// calculation matrix T5 = x*Y3 = t_y1*y2*Y3
MultMatrixNum(T5, x1[0][0], Y3, n, n);

// calculation matrix T6 = T4 + T5 = Y3*Y3 + t_y1*y2*Y3
SumMatrix(T6, T4, T5, n, n);

// calculation matrix T7 = T3 + T6 = (y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 +
t_y1*y2*Y3
SumMatrix(T7, T3, T6, n, n);

// calculation vector t1 = t_y2 * T7 = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3
+ t_y1*y2*Y3)
MultMatrix(t1, t_y2, T7, 1, n, n);

// calculation x = t1*y1 = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 +
t_y1*y2*Y3)*y1
MultMatrix(x, t1, y1, 1, n, 1);

QueryPerformanceCounter(&end);
double elapsedTime = (double)(end.QuadPart - start.QuadPart) / frequency.QuadPart
* 1000;
printf("\nElapsed time : %f milliseconds\n", elapsedTime);

if (out == 1)
{
    PrintMatrixToFile(file, (char*)"Matrix y1*t_y2 = ", T1, n, n);

    PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2) = ", T2, n, n);

    PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2)*(y1*t_y2) = ",
T3, n, n);

    PrintMatrixToFile(file, (char*)"Matrix Y3*Y3 = ", T4, n, n);

    PrintMatrixToFile(file, (char*)"t_y1*y2 = ", x1, 1, 1);

    PrintMatrixToFile(file, (char*)"Matrix t_y1*y2*Y3 = ", T5, n, n);

    PrintMatrixToFile(file, (char*)"Matrix Y3*Y3 + t_y1*y2*Y3 = ", T6, n, n);

    PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3
+ t_y1*y2*Y3 = ", T7, n, n);

    PrintMatrixToFile(file, (char*)"Vector t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) +
Y3*Y3 + t_y1*y2*Y3) = ", t1, 1, n);
}

PrintMatrixToFile(file, (char*)"x = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 +
t_y1*y2*Y3)*y1 = ", x, 1, 1);
printf("\nx = ");
PrintMatrix(x, 1, 1);

if (n == 1)
{
    printf("Checking the result (for n = 1)");
}

```

```

        double res = t_y2[0][0] * ((y1[0][0] * t_y2[0][0]) * (y1[0][0] * t_y2[0][0])
* (y1[0][0] * t_y2[0][0]) + Y3[0][0]*Y3[0][0] + t_y1[0][0] * y2[0][0] * Y3[0][0]) *
y1[0][0];
        printf("\nx = %.2lf\n", res);
    }

    DeleteMatrix(y1, n, 1);
    DeleteMatrix(y2, n, 1);
    DeleteMatrix(Y3, n, n);

    DeleteMatrix(t_y1, 1, n);
    DeleteMatrix(t_y2, 1, n);

    DeleteMatrix(T1, n, n);
    DeleteMatrix(T2, n, n);
    DeleteMatrix(T3, n, n);
    DeleteMatrix(T4, n, n);
    DeleteMatrix(T5, n, n);
    DeleteMatrix(T6, n, n);
    DeleteMatrix(T7, n, n);

    DeleteMatrix(x1, 1, 1);

    DeleteMatrix(t1, 1, n);

    DeleteMatrix(x, 1, 1);

    fclose(file);

    return 0;
}

```

```

// Додаток Б – MPI програма
#define _CRT_SECURE_NO_WARNINGS
#include "mpi.h"
#include <stdio.h>
#include <stdlib.h>
#include <malloc.h>
#include <time.h>
#include <windows.h>

double** CreateMatrix(int m, int n)
{
    double** a = (double**)malloc(m * sizeof(double*));
    for (int i = 0; i < m; i++)
        a[i] = (double*)malloc(n * sizeof(double));
    return a;
}

void DeleteMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        free(a[i]);
    free(a);
}

void FillMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            a[i][j] = rand() % 10 + 1;
            //a[i][j] = 1;
        }
}

void InputMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("Enter [%d][%d] element = ", i, j);
            fflush(stdout);
            scanf_s("%lf", &a[i][j]));
        }
        printf("\n");
    }
}

void PrintMatrix(double** a, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
            printf("%8.2f\t", a[i][j]);
        printf("\n");
    }
    printf("\n");
}

void PrintMatrixToFile(FILE* f, char* title, double** a, int m, int n)
{
    fprintf(f, title);
    fprintf(f, "\n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)

```

```

        fprintf(f, "%.2f\t", a[i][j]);
        fprintf(f, "\n");
    }
    fprintf(f, "\n");
}

// c = a * b, a - mxn, b - nxk, c - mxk
double** MultMatrix(double** c, double** a, double** b, int m, int n, int k)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < k; j++)
        {
            c[i][j] = 0;
            for (int l = 0; l < n; l++)
                c[i][j] = c[i][j] + a[i][l] * b[l][j];
        }
    return c;
}

// b = k * a
double** MultMatrixNum(double** b, double k, double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            b[i][j] = a[i][j] * k;
    return b;
}

// c = a + b
double** SumMatrix(double** c, double** a, double** b, int m, int n)
{
    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    return c;
}

// vector transposition (column-to-row conversion)
void TVector(double** b, double** a, int n)
{
    for (int i = 0; i < n; i++)
        b[0][i] = a[i][0];
}

int main(int argc, char* argv[])
{
    srand((unsigned int)time(NULL));

    MPI_Init(NULL, NULL);

    LARGE_INTEGER frequency, start, end;

    int rank, size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size < 2)
    {
        printf("This program requires at least two processes !\n");
        MPI_Abort(MPI_COMM_WORLD, 1);
    }

    int n;
    int in;
    int out;

```

```

if (rank == 0)
{
    printf("Enter the matrix size n : ");
    fflush(stdout);
    scanf_s("%d", &n);

    // Процес з рангом 0 відправляє n процесу з рангом 1
    MPI_Send(&n, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

    const char* fileName = "Results_0.txt";
    FILE* file = fopen(fileName, "w");
    if (file == NULL)
    {
        printf("\nError opening file. Check the path and permissions !\n");
        return 1;
    }

    printf("\nSelect an input option : 1 - manual input, else - random input :
");
    fflush(stdout);
    scanf_s("%d", &in);
    printf("\n");

    double** y1 = CreateMatrix(n, 1);
    double** y2 = CreateMatrix(n, 1);
    double** Y3 = CreateMatrix(n, n);

    if (in == 1)
    {
        // manual input vector y1
        printf("Enter the elements of vector y1 :\n");
        InputMatrix(y1, n, 1);
        PrintMatrixToFile(file, (char*)"Vector y1 = ", y1, n, 1);

        // manual input vector y2
        printf("Enter the elements of vector y2 :\n");
        InputMatrix(y2, n, 1);
        PrintMatrixToFile(file, (char*)"Vector y2 = ", y2, n, 1);

        // manual input matrix Y3
        printf("Enter the elements of matrix Y3 :\n");
        InputMatrix(Y3, n, n);
        PrintMatrixToFile(file, (char*)"Matrix Y3 = ", Y3, n, n);
    }
    else
    {
        printf("Random input... \n"); \

        // random input vector y1
        FillMatrix(y1, n, 1);
        PrintMatrixToFile(file, (char*)"Vector y1 = ", y1, n, 1);

        // random input vector y2
        FillMatrix(y2, n, 1);
        PrintMatrixToFile(file, (char*)"Vector y2 = ", y2, n, 1);

        // random input matrix Y3
        FillMatrix(Y3, n, n);
        PrintMatrixToFile(file, (char*)"Matrix Y3 = ", Y3, n, n);
    }

    for (int i = 0; i < n; ++i)
        MPI_Send(y1[i], 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);

    for (int i = 0; i < n; ++i)
        MPI_Send(y2[i], 1, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);
}

```

```

for (int i = 0; i < n; ++i)
    MPI_Send(Y3[i], n, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD);

double** t_y2 = CreateMatrix(1, n);

double** T1 = CreateMatrix(n, n);
double** T2 = CreateMatrix(n, n);
double** T3 = CreateMatrix(n, n);
double** T6 = CreateMatrix(n, n);
double** T7 = CreateMatrix(n, n);

double** t1 = CreateMatrix(1, n);

double** x = CreateMatrix(1, 1);

printf("\nWrite intermediate results to a file? : 1 - yes, else - no : ");
fflush(stdout);
scanf_s("%d", &out);

MPI_Send(&out, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);

QueryPerformanceFrequency(&frequency);
QueryPerformanceCounter(&start);

TVector(t_y2, y2, n);

// calculation matrix T1 = y1*t_y2
MultMatrix(T1, y1, t_y2, n, 1, n);

// calculation matrix T2 = T1*T1 = (y1*t_y2)*(y1*t_y2)
MultMatrix(T2, T1, T1, n, n, n);

// calculation matrix T3 = T2*T1 = T1*T1*T1 = (y1*t_y2)*(y1*t_y2)*(y1*t_y2)
MultMatrix(T3, T2, T1, n, n, n);

for (int i = 0; i < n; ++i)
    MPI_Recv(T6[i], n, MPI_DOUBLE, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

// calculation matrix T7 = T3 + T6 = (y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 +
t_y1*y2*Y3
SumMatrix(T7, T3, T6, n, n);

// calculation vector t1 = t_y2 * T7 = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) +
Y3*Y3 + t_y1*y2*Y3)
MultMatrix(t1, t_y2, T7, 1, n, n);

// calculation x = t1*y1 = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 +
t_y1*y2*Y3)*y1
MultMatrix(x, t1, y1, 1, n, 1);

QueryPerformanceCounter(&end);
double elapsedTime = static_cast<double>(end.QuadPart - start.QuadPart) /
frequency.QuadPart * 1000;
printf("\nElapsed Time: %f milliseconds on %d process\n", elapsedTime,
rank);

if (out == 1)
{
    PrintMatrixToFile(file, (char*)"Matrix y1*t_y2 = ", T1, n, n);

    PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2) = ", T2,
n, n);

    PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2)*(y1*t_y2)
= ", T3, n, n);

```

```

        PrintMatrixToFile(file, (char*)"Matrix (y1*t_y2)*(y1*t_y2)*(y1*t_y2)
+ Y3*Y3 + t_y1*y2*Y3 = ", T7, n, n);

        PrintMatrixToFile(file, (char*)"Vector
t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) + Y3*Y3 + t_y1*y2*Y3) = ", t1, 1, n);
    }

    PrintMatrixToFile(file, (char*)"x = t_y2*((y1*t_y2)*(y1*t_y2)*(y1*t_y2) +
Y3*Y3 + t_y1*y2*Y3)*y1 = ", x, 1, 1);
    printf("\nx = ");
    PrintMatrix(x, 1, 1);

    DeleteMatrix(y1, n, 1);
    DeleteMatrix(y2, n, 1);
    DeleteMatrix(Y3, n, n);

    DeleteMatrix(t_y2, 1, n);

    DeleteMatrix(T1, n, n);
    DeleteMatrix(T2, n, n);
    DeleteMatrix(T3, n, n);

    DeleteMatrix(T6, n, n);
    DeleteMatrix(T7, n, n);

    DeleteMatrix(t1, 1, n);

    DeleteMatrix(x, 1, 1);

    fclose(file);
}
else if (rank == 1)
{
    // Процес з рангом 1 отримує n від процесу з рангом 0
    MPI_Recv(&n, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    const char* fileName = "Results_1.txt";
    FILE* file = fopen(fileName, "w");
    if (file == NULL)
    {
        printf("\nError opening file. Check the path and permissions !\n");
        return 1;
    }

    double** y1 = CreateMatrix(n, 1);
    double** y2 = CreateMatrix(n, 1);
    double** Y3 = CreateMatrix(n, n);

    for (int i = 0; i < n; ++i)
        MPI_Recv(y1[i], 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    for (int i = 0; i < n; ++i)
        MPI_Recv(y2[i], 1, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    for (int i = 0; i < n; ++i)
        MPI_Recv(Y3[i], n, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);

    MPI_Recv(&out, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    double** t_y1 = CreateMatrix(1, n);
    double** T4 = CreateMatrix(n, n);

```



```

double** T5 = CreateMatrix(n, n);
double** T6 = CreateMatrix(n, n);

double** x1 = CreateMatrix(1, 1);

QueryPerformanceFrequency(&frequency);
QueryPerformanceCounter(&start);

TVector(t_y1, y1, n);

// calculation matrix T4 = Y3*Y3
MultMatrix(T4, Y3, Y3, n, n, n);

// calculation x1 = t_y1*y2
MultMatrix(x1, t_y1, y2, 1, n, 1);

// calculation matrix T5 = x*Y3 = t_y1*y2*Y3
MultMatrixNum(T5, x1[0][0], Y3, n, n);

// calculation matrix T6 = T4 + T5 = Y3*Y3 + t_y1*y2*Y3
SumMatrix(T6, T4, T5, n, n);

for (int i = 0; i < n; ++i)
    MPI_Send(T6[i], n, MPI_DOUBLE, 0, 0, MPI_COMM_WORLD);

QueryPerformanceCounter(&end);
double elapsedTime = static_cast<double>(end.QuadPart - start.QuadPart) /
frequency.QuadPart * 1000;
printf("\nElapsed Time: %f milliseconds on %d process\n", elapsedTime,
rank);

if (out == 1)
{
    PrintMatrixToFile(file, (char*)"Matrix Y3*Y3 = ", T4, n, n);

    PrintMatrixToFile(file, (char*)"t_y1*y2 = ", x1, 1, 1);

    PrintMatrixToFile(file, (char*)"Matrix t_y1*y2*Y3 = ", T5, n, n);

    PrintMatrixToFile(file, (char*)"Matrix Y3*Y3 + t_y1*y2*Y3 = ", T6, n,
n);
}

DeleteMatrix(y1, n, 1);
DeleteMatrix(y2, n, 1);
DeleteMatrix(Y3, n, n);

DeleteMatrix(t_y1, 1, n);

DeleteMatrix(T4, n, n);
DeleteMatrix(T5, n, n);
DeleteMatrix(T6, n, n);

DeleteMatrix(x1, 1, 1);

fclose(file);
}

MPI_Finalize();

return 0;
}

```

ДОДАТОК В – теоретичні відомості про матриці

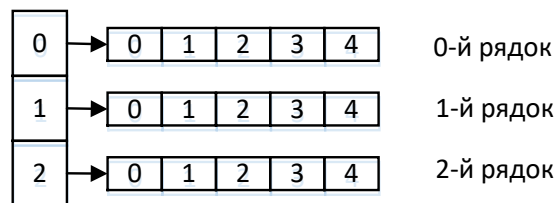
Матриця – це математичний об’єкт, записаний у вигляді прямокутної таблиці чисел, над яким можна виконувати операції додавання, віднімання, множення і множення на скаляр.

Матрицею **A** розмірності $m \times n$ називається таблиця чисел, яка складається з m рядків та n стовпців.

$$A = A_{m \times n} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}$$

Числа, що складають матрицю, називаються її елементами і нумеруються двома індексами, які вказують на номер рядка та стовпця на перетині яких розташований даний елемент. Тобто, елемент a_{ij} міститься в i -му рядку та j -му стовпці матриці **A**.

Матриця яка складається з одного стовпця чи одного рядка, називається відповідно матриця-стовпець чи матриця-рядок. Матрицю-стовпець і матрицю-рядок називають також векторами. Матриця, у якій кількість рядків і стовпців рівна, називається квадратною матрицею. Для розміщення елементів матриці у пам’яті комп’ютера використаємо таку структуру даних як динамічний двовимірний масив. Наприклад розмістимо в пам’яті матрицю 3x5:



При цьому буде виділено пам’ять під кожний рядок матриці окремо, тобто буде утворено три різні одновимірні масиви. Адреси нульових елементів цих масивів зберігатимуться в допоміжному масиві (пам’ять під нього слід виділити заздалегідь). Елементами цього масиву будуть вказівники.

Функція **CreateMatrix()** виділяє пам’ять під динамічний двовимірний масив розміром m на n елементів:

```
double** CreateMatrix(int m, int n)
{
    double** a = (double**)malloc(m * sizeof(double*));
    for (int i = 0; i < m; i++)
        a[i] = (double*)malloc(n * sizeof(double));
    return a;
}
```

Функція **DeleteMatrix()** звільняє виділену пам’ять під динамічний двовимірний масив розміром m на n елементів:

```
void DeleteMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
        free(a[i]);
    free(a);
}
```

Функція **FillMatrix()** заповнює динамічний двовимірний масив розміром m на n елементів випадковими значеннями, згенерованими за допомогою **rand()**:

```
void FillMatrix(double** a, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
```

```

        for (j = 0; j < n; j++)
        {
            a[i][j] = rand() % 10;
        }
    }

```

Функція `InputMatrix()` вводить елементи динамічного двовимірного масиву розміром m на n елементів з клавіатури:

```

void InputMatrix(double** a, int m, int n)
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("Enter [%d][%d] element = ", i, j);
            scanf_s("%lf", &(a[i][j]));
        }
        printf("\n");
    }
}

```

Функція `PrintMatrix()` виводить елементи динамічного двовимірного масиву розміром m на n елементів на екран:

```

void PrintMatrix(double** a, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
    {
        for (j = 0; j < n; j++)
            printf("%8.2f\t", a[i][j]);
        printf("\n");
    }
    printf("\n");
}

```

Сумою матриць A і B є матриця C , елементи якої рівні сумі відповідних елементів матриць A і B . Додавати між собою можна тільки матриці однакового розміру.

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \cdots & \cdots & \cdots & \cdots \\ c_{m1} & c_{m2} & \cdots & c_{mn} \end{pmatrix}$$

$$c_{ij} = a_{ij} + b_{ij}$$

Функція `SumMatrix()` обраховує суму двовимірних масивів a і b розміром m на n елементів, результат зберігає у масив c :

```

// c = a + b
double** SumMatrix(double** c, double** a, double** b, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + b[i][j];
        }
    return c;
}

```

}

Добутком матриці $\mathbf{A}$ на число k називається матриця $\mathbf{B} = k * \mathbf{A}$ того ж розміру, отримана з початкової множенням на задане число всіх її елементів:

$$k * \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} = \begin{pmatrix} k * a_{11} & k * a_{12} & \cdots & k * a_{1n} \\ k * a_{21} & k * a_{22} & \cdots & k * a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ k * a_{m1} & k * a_{m2} & \cdots & k * a_{mn} \end{pmatrix}$$

$$b_{ij} = k * a_{ij}$$

Функція `MultMatrixNum()` множить число k на двовимірний масив $\mathbf{a}$ розміром m на n елементів, результат зберігає у масив $\mathbf{b}$:

```
// b = k * a
double** MultMatrixNum(double** b, double k, double** a, int m, int n)
{
    int i, j;
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
            b[i][j] = a[i][j] * k;
    return b;
}
```

Результатом **множення матриць** $\mathbf{A}_{m \times n}$ та $\mathbf{B}_{n \times k}$ буде матриця $\mathbf{C}_{m \times k}$ така, що елемент матриці $\mathbf{C}$, що знаходяться в i -тому рядку та j -тому стовпчику (c_{ij}), дорівнює сумі добутків елементів i -того рядку матриці $\mathbf{A}$ на відповідні елементи j -того стовпця матриці $\mathbf{B}$:

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} * \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1k} \\ b_{21} & b_{22} & \cdots & b_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nk} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1k} \\ c_{21} & c_{22} & \cdots & c_{2k} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mk} \end{pmatrix}$$

$$c_{ij} = a_{i1} * b_{1j} + a_{i2} * b_{2j} + \cdots + a_{in} * b_{nj}$$

Дві матриці можна перемножити між собою тоді і тільки тоді, коли кількість стовпців першої матриці дорівнює кількості рядків другої матриці.

Часткові випадки – множення матриці на матрицю стовпець,

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix} * \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix} = \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_m \end{pmatrix}$$

$$c_i = a_{i1} * b_1 + a_{i2} * b_2 + \cdots + a_{in} * b_n$$

множення матриці рядка на матрицю

$$\begin{pmatrix} a_1 & a_2 & \cdots & a_m \end{pmatrix} * \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{m1} & b_{m2} & \cdots & b_{mn} \end{pmatrix} = \begin{pmatrix} c_1 & c_2 & \cdots & c_n \end{pmatrix}$$

$$c_i = a_1 * b_{1i} + a_2 * b_{2i} + \cdots + a_m * b_{mi}$$

множення матриці стовпця на матрицю рядок

$$\begin{pmatrix} a_1 \\ a_2 \\ \dots \\ a_m \end{pmatrix} * (b_1 \quad b_2 \quad \dots \quad b_m) = \begin{pmatrix} c_{11} & c_{12} & \dots & c_{1m} \\ c_{21} & c_{22} & \dots & c_{2m} \\ \dots & \dots & \dots & \dots \\ c_{m1} & c_{m2} & \dots & c_{mm} \end{pmatrix}$$

$$c_{ij} = a_i * b_j$$

і множення рядка на стовпець.

$$(a_1 \quad a_2 \quad \dots \quad a_n) * \begin{pmatrix} b_1 \\ b_2 \\ \dots \\ b_n \end{pmatrix} = (c)$$

$$c = a_i * b_i + a_i * b_i + \dots + a_n * b_n$$

Функція **MultMatrix()** множить двовимірний масив **a** розміром **m** на **n** елементів на масив **b** розміром **n** на **k** елементів результат зберігає у масив **c** розміром **m** на **k** елементів:

```
// c = a * b, a - mxn, b - nxk, c - mxk
double** MultMatrix(double** c, double** a, double** b, int m, int n,
int k)
{
    int i, j, l;
    for (i = 0; i < m; i++)
        for (j = 0; j < k; j++)
        {
            c[i][j] = 0;
            for (l = 0; l < n; l++)
                c[i][j] = c[i][j] + a[i][l] * b[l][j];
        }
    return c;
}
```